

Windowed & Shifted-Window Attention.

Global ViT attention on an $h \times w$ patch grid costs $O((hw)^2)$ — quadratic in image *area*. Swin instead attends inside non-overlapping $M \times M$ windows, making the cost *linear* in area for fixed M . The price is that fixed windows never talk to each other; the fix is to *shift* the windows on alternate layers. Below we derive both complexities, compute the saving for a real 56×56 grid, and draw the regular and shifted partitions. Every number on these pages is computed in full — nothing summarised.

THE EQUATION

$$\Omega(\text{MSA}) = 4hw d^2 + 2(hw)^2 d \quad \implies \quad \Omega(\text{W-MSA}) = 4hw d^2 + 2M^2 hw d$$

Worked example. $h = w = 56$, $M = 7$, $d = 96$: a $64\times$ reduction in the attention term; full table for grids 56, 28, 14, plus heatmaps of the regular and shifted 8×8 partitions.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The claim: global attention is $O((hw)^2)$	2
2	Windowing: restrict attention to local $M \times M$ tiles	3
3	Worked example: $h = w = 56$, $M = 7$, $d = 96$	4
4	The problem windowing creates — and the shifted-window fix	4
5	Implementation: cyclic shift + masking	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 The claim: global attention is $O((hw)^2)$

A vision transformer splits an image into a grid of $h \times w$ patches and flattens it to $N = hw$ tokens of dimension d . Plain (global) multi-head self-attention then runs scaled dot-product attention over *all* N tokens:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q, K, V \in \mathbb{R}^{hw \times d} \quad (1)$$

The score matrix $S = QK^\top / \sqrt{d}$ is $hw \times hw$: one entry for every *ordered pair of patches*. Counting the four linear projections (Q, K, V , and output) plus the two attention products, a standard MSA block on the grid costs

$$\Omega(\text{MSA}) = \underbrace{4hw d^2}_{\text{projections}} + \underbrace{2(hw)^2 d}_{\text{attention}} \quad (2)$$

Symbol	Shape	Meaning
$h \times w$	—	patch grid (rows $\times$ cols)
$N = hw$	—	number of tokens (image <i>area</i> in patches)
d	—	channel / model dimension
Q, K, V	$hw \times d$	queries, keys, values
$S = QK^\top / \sqrt{d}$	$hw \times hw$	global scores — the costly object
M	—	window side (window holds M^2 tokens)

The first term, $4hw d^2$, is linear in area hw (four $d \times d$ projections applied to hw rows). The second, $2(hw)^2 d$, is the $QK^\top$ and PV products and is *quadratic in area*. As soon as the grid is moderately large, that quadratic term dominates.

The expensive part of attention is the all-pairs score matrix. On a 56×56 grid that is 3136 patches, so $3136^2 \approx 9.8$ million pairs — before a single non-linearity. Doubling the image side quadruples the patch count and *quadruples again* the pair count: a $16\times$ blow-up for $2\times$ the resolution. Vision is exactly where this hurts most, because high-resolution images mean huge hw .

2 Windowing: restrict attention to local $M \times M$ tiles

Swin partitions the grid into non-overlapping windows of $M \times M$ patches. There are

$$\frac{h}{M} \times \frac{w}{M} = \frac{hw}{M^2} \quad \text{windows,}$$

and attention runs *independently inside each window* over its M^2 tokens. One window's attention term costs $2(M^2)^2 d = 2M^4 d$. Summing over all hw/M^2 windows:

$$\underbrace{\frac{hw}{M^2}}_{\text{\#windows}} \times \underbrace{2M^4 d}_{\text{per window}} = 2M^2 hw d.$$

The projections are unchanged ($4hw d^2$, every token still gets Q, K, V). So the windowed block (W-MSA) costs

$$\Omega(\text{W-MSA}) = \underbrace{4hw d^2}_{\text{projections}} + \underbrace{2M^2 hw d}_{\text{attention}} \quad (3)$$

For *fixed* M , the attention term is now $2M^2 hw d = O(hw)$ — **linear in image area**, not quadratic. The whole win is the swap $(hw)^2 \rightarrow M^2 hw$ inside the attention term.

The attention saving factor is exactly the number of windows:

$$\frac{2(hw)^2 d}{2M^2 hw d} = \frac{hw}{M^2} = \text{\#windows.}$$

Every window you cut the grid into divides the all-pairs attention cost by one. With M held fixed, more resolution means more windows means a bigger relative saving — the opposite of global attention, which gets relatively *worse*.

3 Worked example: $h = w = 56$, $M = 7$, $d = 96$

The first Swin stage operates on a 56×56 grid, so $hw = 3136$ tokens, with $d = 96$ and window side $M = 7$ ($M^2 = 49$ tokens per window, $3136/49 = 64$ windows).

Pure attention pairs (the bare $QK^\top$ scale): global is $hw^2 = 3136^2 = 9,834,496$ pairs. Windowed is $64 \text{ windows} \times 49^2 = 64 \times 2401 = 153,664$ pairs. Ratio $9,834,496/153,664 = \mathbf{64}$, i.e. the window count, as the key idea predicts.

Full block FLOPs from Eqs. (2)–(3) with $d = 96$:

$$\begin{aligned}\Omega(\text{MSA}) &= 4(3136)(96^2) + 2(3136)^2(96) = 115,605,504 + 1,888,223,232 = 2,003,828,736, \\ \Omega(\text{W-MSA}) &= 4(3136)(96^2) + 2(49)(3136)(96) = 115,605,504 + 29,503,488 = 145,108,992.\end{aligned}$$

The block as a whole is $2,003,828,736/145,108,992 = \mathbf{13.8\times}$ cheaper; the *attention term alone* shrinks $1,888,223,232/29,503,488 = \mathbf{64\times}$. (The ratio of full blocks is smaller than 64 only because the shared $4hwd^2$ projection term does not change.)

3.1 How the saving grows down the Swin pyramid

Later stages shrink the grid ($56 \rightarrow 28 \rightarrow 14$) but keep $M = 7$. Fewer windows means a smaller relative saving — exactly because there are fewer windows to divide by:

Grid	hw	#windows hw/M^2	attn term saving	$\Omega(\text{MSA})/\Omega(\text{W-MSA})$
56×56	3136	64	$64\times$	$13.81\times$
28×28	784	16	$16\times$	$4.05\times$
14×14	196	4	$4\times$	$1.61\times$

The attention-term saving is always the window count hw/M^2 ; the full-block ratio is damped by the fixed projection cost (with $d = 96$). Either way, Swin pays linear, not quadratic, area cost.

4 The problem windowing creates — and the shifted-window fix

Independent windows have a fatal flaw: *no information ever crosses a window boundary*. A patch in the top-left window cannot, at any depth, attend to a patch in the top-right window. The receptive field is frozen at $M \times M$ forever. Swin's fix is to **shift the window grid by $\lfloor M/2 \rfloor$** on every *second* layer (W-MSA $\rightarrow$ SW-MSA $\rightarrow$ W-MSA $\dots$). The shifted partition straddles the previous boundaries, so two patches that were in different windows now share one — cross-window connections form, and after two layers every patch can reach a $2M \times 2M$ neighbourhood.

Below, an 8×8 grid is cut into four 4×4 windows ($M = 4$, shift = 2). Cell colour = window id $\{0, 1, 2, 3\}$. **Left:** the regular partition. **Right:** the partition after shifting by 2 — the same physical patch now lands in a different window.

0	0	0	0	1	1	1	1	3	3	2	2	2	2	3	3
0	0	0	0	1	1	1	1	3	3	2	2	2	2	3	3
0	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1
0	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1
2	2	2	2	3	3	3	3	1	1	0	0	0	0	1	1
2	2	2	2	3	3	3	3	1	1	0	0	0	0	1	1
2	2	2	2	3	3	3	3	3	3	2	2	2	2	3	3
2	2	2	2	3	3	3	3	3	3	2	2	2	2	3	3

Regular (left): four clean 4×4 blocks. Shifted (right): the window grid is rolled by 2 so the central 4×4 block (window 0) now spans what were four different windows — those patches can finally attend to one another. The corners that wrap around (window 3) are non-adjacent and must be masked, see below.

5 Implementation: cyclic shift + masking

A naive shift produces partial windows at the edges (smaller than $M \times M$), which would need padding and more windows. Swin avoids this with a **cyclic shift**: `torch.roll` the whole feature map by $(-\lfloor M/2 \rfloor, -\lfloor M/2 \rfloor)$, then re-partition into the *same* hw/M^2 full windows. This keeps the window count — and hence the FLOP count of Eq. (3) — *exactly the same* as W-MSA. The catch: a shifted window may now group patches that wrap around from opposite edges of the image and are not truly neighbours (window 3's four corner blocks above). An **attention mask** sets the scores between such non-adjacent sub-groups to $-\infty$ before the softmax, so they contribute zero weight. After attention, a reverse roll restores the original layout.

Shifting is *not* free overlap. The cyclic-shift trick deliberately keeps the number of windows fixed so SW-MSA costs the same as W-MSA — but it then *must* mask the wrapped-around pairs. Forget the mask and corner patches attend to unrelated patches on the far edge of the image, silently corrupting features. The mask is what makes the cheap cyclic shift correct.

6 Where this sits in the track

Idea	Cost in area hw	Where
Global ViT attention	$O((hw)^2)$ quadratic	Phase 6, ViT
Windowed (W-MSA)	$O(M^2 hw)$ linear, no cross-window	this topic
Shifted (SW-MSA)	$O(M^2 hw)$ linear, cross-window via shift+mask	this topic
Patch merging	builds the multi-scale pyramid Swin runs on	Topic 2 (next)

Windowing gives Swin linear cost; shifting restores global information flow. The next topic shows how Swin stacks these blocks across a *pyramid* of resolutions via patch merging, which is what makes it a general-purpose vision backbone.

A Reproducible (NumPy)

Inputs: $d = 96$, $M = 7$; grids 56, 28, 14; partition demo on an 8×8 grid with $M = 4$, shift 2.

```
import numpy as np
d, M = 96, 7
for s in (56, 28, 14):
    hw = s*s
    msa = 4*hw*d*d + 2*hw*hw*d
    wmsa = 4*hw*d*d + 2*M*M*hw*d
    print(s, hw, hw/(M*M), 2*hw*hw*d/(2*M*M*hw*d), round(msa/wmsa, 2))
# 56 3136 64 64 13.81 | 28 784 16 16 4.05 | 14 196 4 4 1.61
print(3136**2, 64*49*49)          # 9834496 153664 (global vs windowed pairs, ratio 64)

# --- 8x8 partition: regular and cyclic-shifted (M=4, shift=2) ---
G, M, s = 8, 4, 2
reg = np.array([(r//M)*2 + (c//M) for c in range(G)] for r in range(G))
shf = np.array([(((r-s)%G)//M)*2 + (((c-s)%G)//M) for c in range(G)] for r in range(G))
print(reg); print(shf)           # match the two heatmaps
```